# ASR700 Autonomous Systems and Robotics: Project Report

Christinah Makocheredze  Student number: 402110898
GitHub (full commit history): https://github.com/TinahMak/asr700-intelligent-robotic-systems

## 1. Introduction and set-up

This project builds an autonomous mobile-robot system in two parts. Component A takes one robot from raw sensing to closed-loop motion: it estimates its pose, builds an occupancy map from LiDAR, detects obstacles with a camera, plans a path with Dijkstra and A*, and follows that path with a PID heading controller. Component B scales up to a team of four robots that agree on a common reference point through average consensus, share eight tasks using particle swarm optimisation (PSO), and use a neural network trained on LiDAR data they collected themselves to recognise what is in front of them. A final experiment compares centralised and distributed coordination as the team grows.

All work was done in Webots R2025a (Cyberbotics, 2025) on Windows 11 with controllers and analysis scripts in Python 3.12.10 (NumPy, SciPy, OpenCV, Matplotlib); ROS and Gazebo were not used. The robot is the TurtleBot3 Burger, a differential-drive base with wheel radius 0.033 m, wheel separation 0.160 m and a maximum wheel speed of 6.67 rad/s, which gives a top speed of about 0.22 m/s and a top turn rate on the spot of about 2.75 rad/s. Its LDS-01 LiDAR returns 360 beams over 360 degrees with a range of 3.5 m. The arena is 4 x 4 m with six 0.30 m wooden boxes. The arena walls had to be raised from 0.1 m to 0.5 m because the original walls were below the LiDAR scan plane and therefore invisible to it.

The repository is organised into worlds/ (Webots .wbt files), controllers/ (the navigation, data-collection and swarm controllers, each with a shared odometry module), notebooks/ (offline Python scripts for planning, PID tuning, PSO, the neural network and all plots), data/ (saved maps, paths, the LiDAR dataset and network weights) and report/ (figures and the text files of results quoted below). README.txt lists the run commands and run times. The history is a sequence of small commits, from the project skeleton through wheel odometry, compass fusion, mapping, planning, control, the multi-robot world, PSO, the classifier and the coordination study, so the development of each part can be followed step by step.

## 2. Component A: single robot

### 2.1 Perception and mapping

**LiDAR geometry.** The beam ordering of the LDS-01 in Webots was not obvious from the documentation, so it was found by saving a scan at a known pose and comparing it with the world (report/scan_start.png). Beam 0 points backwards and the index runs clockwise, so the angle of beam *i* in the robot frame is $\theta_i = \pi - 2\pi i/360$; beam 180 points straight ahead and beam 90 to the left. This matters because every hit is projected into the world as $(x + r\cos(\theta + \theta_i), y + r\sin(\theta + \theta_i))$. With the wrong sign the map is mirrored left to right, and with the wrong offset it is rotated by 180 degrees, so obstacles would land in the wrong cells and the planner would route the robot through real boxes.

**Pose estimate.** Wheel odometry integrates the encoder increments. Small differences in wheel slip build up mainly in the heading, and a heading error turns every later metre of travel into a sideways position error. After 150 s of wandering, the wheels-only heading differed from the compass by about 0.67 rad (38 degrees). The final estimator keeps the travelled distance from the wheels but takes the heading from the compass, $\theta = \text{atan2}(c_x, c_y)$, because at heading zero the compass reads north along the robot's +y axis. Figure 1 shows the effect: the wheels-only trajectory leaves the arena (down to $y \approx -2.4$ m) and its map is smeared, while the fused trajectory stays inside the walls and gives a sharp map. At t = 152 s the fused position was (−0.56, −1.77) m against a true position of about (−0.56, −1.80) m.

**Occupancy grid.** The arena is covered by 80 x 80 cells of 0.05 m. Each cell stores the log-odds $l = \log(p / (1 - p))$ of being occupied. For each beam the cells along the ray are sampled in half-cell steps and each one receives −0.40 (free); the end cell receives +0.85 (occupied) unless the beam reached its maximum range. Values are clamped to ±5, i.e. p between about 0.007 and 0.993. Log-odds turn the Bayes update into an addition (Thrun, Burgard and Fox, 2005), which is cheap and avoids multiplying many small probabilities. The clamp keeps the map from becoming so certain that it can no longer change. A hit is weighted more strongly than a pass-through, so thin walls are not erased by beams that graze them. Box interiors stay unknown (grey) because no beam ever enters them.

**Camera.** The Burger has no camera, so one was added in the extension slot. Testing saved images showed that only the rotated camera node looks forward (report/camera_orientation_test.png). The OpenCV pipeline converts each image to HSV and thresholds boxes (H 10–20, S ≥ 80, V 40–170) and walls (H 100–125, V < 112), removes dark shadow pixels (V < 45), and cleans the masks with a morphological opening (removes

speckle) and closing (fills holes). The distance of an object comes from the image row *v* where it touches the floor. With the camera 0.12 m above the floor and a focal length of 293 px (1.0 rad horizontal field of view over 320 px), depth = 0.12 × 293 / (v − 120).

**Fusion.** For each image column the camera range is compared with the LiDAR beam at the same bearing. If they agree within a 0.30 m gate, they are combined with inverse-variance weights, $w = 1/\sigma^2$, using $\sigma_{LiDAR}$ = 0.02 m and $\sigma_{camera}$ = 5 % of the range; otherwise the LiDAR is trusted. At 2 m the camera σ is 0.10 m, so the LiDAR weight is 25 times larger and the fused range is essentially the LiDAR range. The camera's real contribution is the class of the obstacle (box or wall), which a single LiDAR range cannot give. In Figure 2 the confirmed camera points follow the LiDAR to within about 0.1–0.2 m; the three rejected points (red) sit where the range jumps between a near object and the far wall, or at the edge of the image, where the floor-contact row is hardest to find.

**Limitations.** Camera depth error grows with the square of the range: at 3 m a one-pixel error in the floor-contact row moves the estimate by about 0.26 m. Colour thresholds depend on lighting and floor texture, and a shadowed box face can be classified as wall. A single fixed forward camera sees only about 57 degrees, so classes are only known for obstacles ahead.

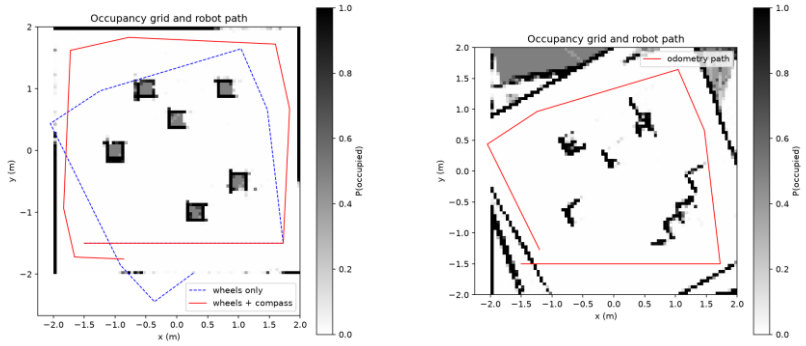



**Figure 1:** *Occupancy map built with compass fusion (left, with both trajectories) versus wheel odometry only (right).*

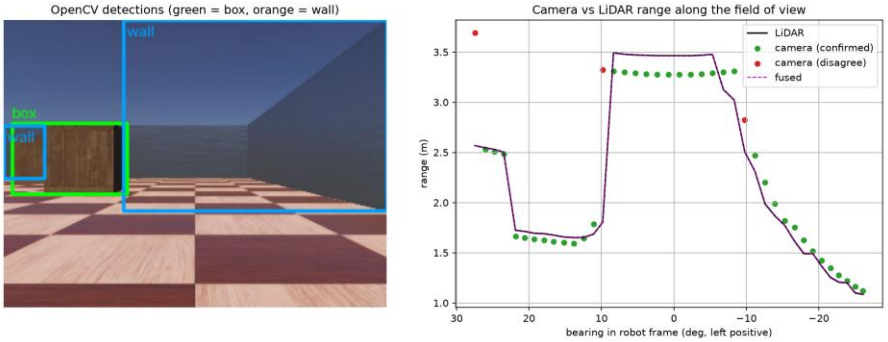



**Figure 2:** *OpenCV detections (left) and camera versus LiDAR range across the camera field of view (right). Red points failed the 0.30 m gate and were not fused.*

## 2.2 Path planning: Dijkstra versus A*

The saved map is turned into a planning grid in which cells with p > 0.5 are obstacles, and every obstacle is inflated by 0.15 m (3 cells), the Burger's radius of about 0.105 m plus a margin, so the robot can be treated as a point. The grid is 8-connected with step costs 1 (straight) and √2 (diagonal) in cell units, and a diagonal move is not allowed if either neighbouring straight cell is blocked, so paths cannot cut the corner of an obstacle. Dijkstra (1959) expands cells in order of cost-so-far g; A* (Hart, Nilsson and Raphael, 1968) orders them by g + h, where h is the octile distance, $h = \Delta_{max} + (\sqrt{2} - 1)\Delta_{min}$. The octile distance is the exact cost on an empty 8-connected grid, so it never overestimates (admissible) and obeys the triangle inequality (consistent). A* therefore returns an optimal path and never needs to reopen a cell, which is why both planners give the same length in every scenario below.

The difference is the search effort. Dijkstra spreads out evenly in all directions and expanded 3658–4441 of the 6400 cells. A* is pulled towards the goal: 588 cells in S1 (south-west to north-east, 7.6 times fewer), 152 in S2 (north-west to south-east, 29 times fewer) and 113 in S3 (west to east, 32 times fewer). S1 is the hardest case for A* because the diagonal crosses the cluster of boxes in the middle of the arena: the straight-line estimate is too optimistic there, and A* has to explore around the boxes before it finds the way through. In S2 and S3 the optimal path stays close to the heuristic, so A* expands little more than the path itself. On an 80 x

80 grid Dijkstra takes about 20 ms and A* 1–5 ms, but A*'s 8–32 times saving in expanded cells would matter for online replanning or for larger maps.

| Scenario | Planner | Path length (m) | Cells expanded |
|---|---|---|---|
| S1 | Dijkstra | 4.45 | 4441 |
| S1 | A* | 4.45 | 588 |
| S2 | Dijkstra | 4.36 | 4390 |
| S2 | A* | 4.36 | 152 |
| S3 | Dijkstra | 3.31 | 3658 |
| S3 | A* | 3.31 | 113 |

*Grid of 80 x 80 cells (0.05 m), obstacles inflated by 0.15 m. Values from report/planning_table.txt; A* run times were 4.5, 1.0 and 0.7 ms against 22.5, 21.5 and 18.9 ms for Dijkstra.*

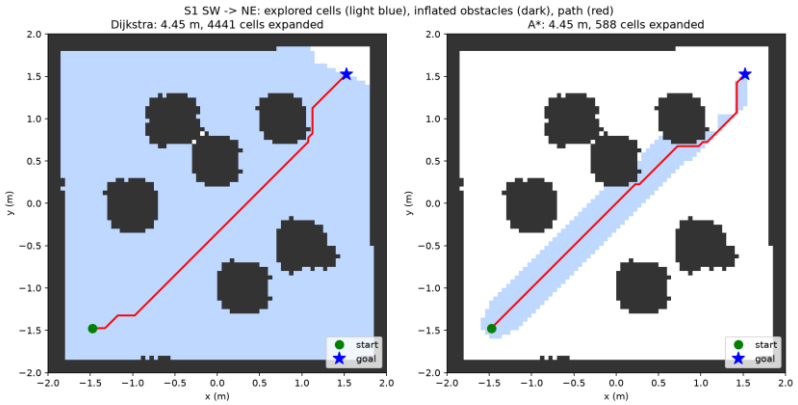



**Figure 3:** *Dijkstra and A* paths with the cells each one expanded (scenario S1).*

## 2.3 Differential-drive kinematics and PID control

**Kinematics.** With wheel speeds $\omega_R$ and $\omega_L$, the forward model is $v = r(\omega_R + \omega_L)/2$ and $\omega = r(\omega_R - \omega_L)/L$. The pose is updated with the heading at the middle of the step, $x \leftarrow x + v\Delta t \cos(\theta + \omega\Delta t/2)$, $y \leftarrow y + v\Delta t \sin(\theta + \omega\Delta t/2)$, $\theta \leftarrow \theta + \omega\Delta t$, which is more accurate on curved paths than using the heading at the start of the step. The inverse model gives $\omega_R = (v + \omega L/2)/r$ and $\omega_L = (v - \omega L/2)/r$. If either wheel would exceed the 6.67 rad/s limit, both are scaled by the same factor. This keeps the ratio $\omega/v$, and therefore the curvature, so the robot slows down but stays on the intended arc; clipping only one wheel would change the turning radius.

**PID.** The controller acts on the heading error e (wrapped to $\pm\pi$) and outputs the turn rate: $u = K_p e + K_i \int e\,dt + K_d\, de/dt$. The proportional term turns towards the target in proportion to the error; the integral removes a steady error caused by a constant disturbance; the derivative damps the motion by reacting to how fast the error is changing. The integral is clamped (anti-windup) so that it cannot keep growing while the wheels are saturated and then cause a large overshoot. The derivative is taken on the error rather than on the measured heading because, when following a path, the target heading changes continuously as the lookahead point moves, and the controller should damp the robot's motion relative to that moving target.

**Results.** The table shows a 90-degree step on the spot, simulated offline with the same kinematics and a first-order motor lag. A low gain ($K_p = 1$) is slow and takes 3.46 s to settle. Raising $K_p$ to 4 or 12 makes the rise much faster but causes 3.5 % and 7.9 % overshoot. Adding derivative action ($K_d = 0.4$) removes the overshoot. Adding integral action ($K_i = 1$) made things worse: 9.7 % overshoot and 7.52 s to settle. Heading control of a differential-drive robot has no steady-state error to remove, because the controller commands the turn rate directly and the heading is its integral; the integral term only stores error during the rise and releases it as overshoot. The final gains ($K_p = 8$, $K_i = 0$, $K_d = 0.8$) came from a grid search scored by settling time plus 0.1 times the percentage overshoot; they settle in 0.90 s with no overshoot. The cost of the derivative term is a ripple of about ±0.3 cm in the path-following error, because differentiating the slightly quantised compass and wheel signals amplifies their noise.

| Controller | Rise (s) | Overshoot (%) | Settle 2 % (s) | IAE |
|---|---|---|---|---|
| P Kp=1 | 1.89 | 0.0 | 3.46 | 1.571 |
| P Kp=4 | 0.51 | 3.5 | 1.28 | 0.676 |
| P Kp=12 | 0.48 | 7.9 | 1.22 | 0.672 |
| PD Kp=4 Kd=0.4 | 0.64 | 0.0 | 1.15 | 0.716 |
| PID Kp=4 Ki=1 Kd=0.4 | 0.54 | 9.7 | 7.52 | 1.226 |
| Tuned Kp=8 Ki=0 Kd=0.8 | 0.54 | 0.0 | 0.90 | 0.663 |

*Heading step of 90 degrees on the spot, from notebooks/pid_tuning.py.*

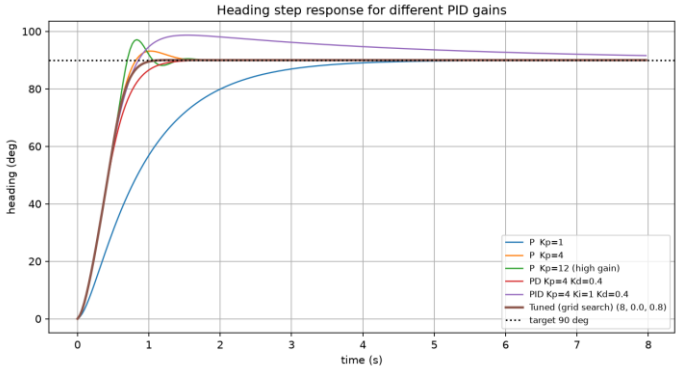



**Figure 4:** *Heading step response (90 degrees) for the different gain sets.*

## 2.4 Closed-loop integration in Webots

In Webots the whole chain runs inside one controller loop. The saved map gives the A* path; at each time step the controller finds the closest path point ahead of the robot and a target point 0.25 m further along the path; the heading error to that point goes into the PID; the PID turn rate and a cruise speed of 0.15 m/s go through the inverse kinematics to the wheel motors; and the fused pose (wheel distance plus compass heading) closes the loop.

The Webots run and the offline simulation agree closely: 29.9 s against 29.5 s to the goal and a mean cross-track error of 1.4 cm against 1.5 cm. The offline model contains only the kinematics, the wheel speed limit and a motor lag, so this agreement shows that a no-slip kinematic model is good enough for the Burger at this speed on a flat floor, and that gains tuned offline transfer directly to the simulator.

Figure 5 shows three regimes. The error starts at 3.5 cm because the robot begins slightly off the first path cell and settles within about 3 s. Along the long straight diagonal (5–15 s) it stays at 0.2–0.5 cm; the fast ripple there is the derivative term amplifying sensor noise. The peaks of 6–7 cm (at about 21 s and 26.5 s) come at the sharp corners of the grid path near (0.7, 0.7) m and (1.45, 1.05) m, not on the straight sections. Because the target is 0.25 m ahead, it moves onto the next segment before the robot reaches the corner, so the robot starts turning early and cuts the corner, which is visible as the red path rounding off the blue one. A shorter lookahead would reduce this but makes the heading command more nervous; smoothing the A* path would be a better fix. The run ends 8.0 cm from the goal in both Webots and the offline simulation because the controller stops on the first step at which the estimated distance falls below the 8 cm tolerance, so the final distance equals the tolerance by design. Time and final distance come from the controller console output; the cross-track errors are computed from the logged path up to the first sample within 8.5 cm of the goal.

| Metric | Offline simulation | Webots |
|---|---|---|
| Time to goal (s) | 29.5 | 29.9 |
| Mean cross-track error (cm) | 1.5 | 1.4 |
| Maximum cross-track error (cm) | 7.1 | 7.0 |
| Final distance to goal (cm) | 8.0 | 8.0 |

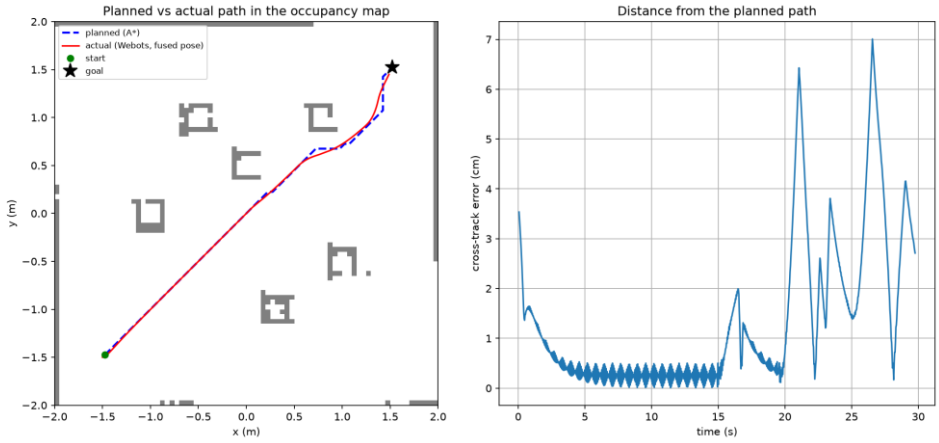



**Figure 5:** *Planned (A*) and actual path in Webots on the occupancy map (left) and cross-track error over time (right).*

# 3. Component B: multi-robot team

## 3.1 Multi-robot world and average consensus

The multi-robot world (worlds/component_b.wbt, generated from the Component A arena by notebooks/make_world_b.py) holds four TurtleBot3 Burgers, each with an Emitter and a Receiver. In the communication test every robot printed the positions of all three others, which confirms that each robot heard the other three.

Each robot *i* holds a state $z_i$ and repeatedly applies

$$z_i \leftarrow z_i + \varepsilon \sum_{j \in N(i)} (z_j - z_i), \quad \text{or in matrix form} \quad z \leftarrow (I - \varepsilon L)\, z,$$

where L is the graph Laplacian (degree on the diagonal, −1 for each link; Olfati-Saber, Fax and Murray, 2007). Every column of L sums to zero, so the sum of the states, and hence their average, never changes; the only fixed point of a connected graph is every robot holding that average. The update is stable if $\varepsilon < 1/d_{max}$, where $d_{max}$ is the largest number of neighbours. How fast the robots agree is set by the second-smallest Laplacian eigenvalue $\lambda_2$ (the algebraic connectivity): disagreement shrinks by a factor of about $1 - \varepsilon\lambda_2$ per round.

Here the four robots form a ring (each talks to two neighbours, $d_{max} = 2$), with $\varepsilon = 0.3 < 0.5$ and one round every 0.2 s. For a ring of four, $\lambda_2 = 2$, so disagreement falls by a factor of $1 - 0.3 \times 2 = 0.4$ per round. The state each robot shares is its position minus its own 0.35 m formation offset, so the robots agree on a common centre while stopping on a 0.35 m ring around it instead of driving into each other. In Webots the states reached the average (0, 0): the disagreement fell below 1 cm after 1.22 s (6.1 rounds) and below 0.1 mm after 2.21 s, a measured contraction of 0.37 per round against the predicted 0.40; by t = 5 s the console showed z = (0.000, 0.000) for every robot, consistent with the geometric decay the analysis predicts. Agreeing on the centre is quicker than getting there: at 5 s the robots, which started in the corners at (±1.5, ±1.5) m and were then at about (±0.99, ±0.99) m, were still about 1.05 m from their targets, because they drive at a cruise speed of 0.15 m/s. Figure 6 shows the end of the run: the four robots have gathered in a square around the arena centre without colliding.

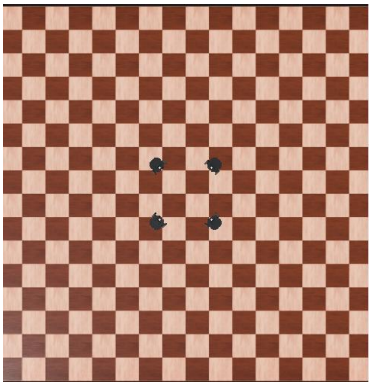


**Figure 6:** *Final formation of the four robots in Webots (top view): after consensus they gather in a square around the arena centre.*

## 3.2 Task allocation with particle swarm optimisation

**Problem.** Four robots near the centre of the arena must share eight tasks spread around it. A solution is encoded as eight real numbers in [0, 4); the floor of number *k* is the robot that does task *k*. Each robot visits

its tasks in nearest-neighbour order, and the fitness to minimise is the longest route plus 0.1 times the total length of all routes. The first term balances the work so the team finishes early, and the second breaks ties in favour of less total driving. There are $4^8 = 65{,}536$ possible assignments.

**Algorithm.** Each particle *i* has a position $x_i$ and velocity $v_i$, updated as $v_i \leftarrow w\, v_i + c_1\, r_1(p_i - x_i) + c_2\, r_2(g - x_i)$ and $x_i \leftarrow x_i + v_i$, with $r_1$, $r_2$ random in [0, 1]. In the formulation of Kennedy and Eberhart (1995), with the inertia weight added by Shi and Eberhart (1998), the inertia w keeps the particle moving in its current direction (exploration), the cognitive term pulls it back towards its own best position $p_i$, and the social term pulls it towards the best position g known to the swarm.

**What went wrong and what helped.** The basic version (30 particles, global best, $w = 0.72$, $c_1 = c_2 = 1.5$, positions clamped at the borders) converged prematurely: after about 600 evaluations all particles had collapsed onto the same assignment and stopped searching. It reached the optimum in only 3 of 20 runs. Three changes helped. First, a ring neighbourhood: each particle follows the best of itself and its two neighbours instead of the global best, so good solutions spread slowly and the swarm stays diverse for longer. This is the same idea as the consensus topology in 3.1, where a sparse ring also spreads information more slowly than a complete graph. Second, an inertia that decays from 0.9 to 0.4 (Shi and Eberhart, 1998), which explores widely early on and refines at the end. Third, reflecting borders, so particles that hit the edge of [0, 4) bounce back instead of piling up on robot 0 or robot 3. With 60 particles, $c_1 = c_2 = 2$ and these changes, PSO reached the optimum in 10 of 20 runs with a mean cost of 3.734, against 4.119 for random search with the same budget.

To be honest about the result: brute force checks all 65,536 assignments in about 1.3 s and is guaranteed to find the optimum, so PSO is not needed at this size. Its value is for larger teams: with 8 robots and 20 tasks there are $8^{20} \approx 10^{18}$ assignments, far beyond brute force, while the PSO cost per evaluation stays the same. The greedy nearest-robot rule (cost 4.844) shows that a simple heuristic leaves a lot of improvement on the table, because it ignores how the work is balanced.

| Method | Evaluations | Mean cost | Best | Worst | Runs reaching optimum |
|---|---|---|---|---|---|
| Brute force (optimum) | 65536 | 3.595 | 3.595 | 3.595 | exact |
| Greedy nearest robot | 4 | 4.844 | 4.844 | 4.844 | no (deterministic) |
| Random search (basic budget) | 1830 | 4.383 | 3.769 | 4.899 | 0/20 |
| Random search (improved budget) | 3660 | 4.119 | 3.595 | 4.310 | 1/20 |
| PSO basic | 1830 | 4.158 | 3.595 | 4.853 | 3/20 |
| PSO improved | 3660 | 3.734 | 3.595 | 4.236 | 10/20 |

*20 runs per method, 60 iterations each; random search is given the same number of evaluations as each PSO variant. Values from report/pso_results.txt.*

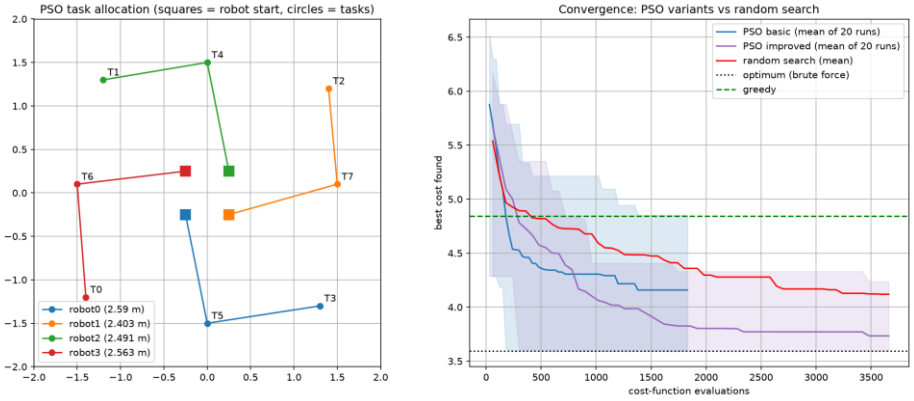



**Figure 7:** *Best allocation found (left) and convergence of PSO versus random search (right).*

## 3.3 Neural network on collected data

**Task and data.** The network classifies what is straight ahead of the robot, *clear* (nothing within 1.2 m), *box* or *wall*, from a raw LiDAR scan. A wandering robot logged 7,201 scans together with its fused pose over 60 simulated minutes, of which 5,863 remained after cleaning (see below). The network only sees 46 ranges (every fourth beam across the front 180 degrees, scaled by the 3.5 m maximum and standardised); it never sees the pose.

**Labels.** Labels were generated automatically rather than by hand. The nearest hit in the front ±10 degrees is projected into the world using the fused pose; if it falls on one of the six boxes (whose positions are read from

the .wbt file) the label is *box*, if it falls near the arena boundary it is *wall*, and if nothing is within 1.2 m it is *clear*. This is fast and consistent, but the labels inherit any pose error. A few centimetres of error matters most near box corners and near the 1.2 m threshold, so some labels there are probably wrong, and the measured accuracy may understate what the network has learnt.

**Model.** A multilayer perceptron 46–32–16–3 with ReLU hidden layers and a softmax output, written in NumPy, with He initialisation, the Adam optimiser, a class-weighted cross-entropy loss (so the rare box class counts more) and early stopping on validation loss (stopped at epoch 46). Backpropagation (Rumelhart, Hinton and Williams, 1986) applies the chain rule backwards through the network: the gradient of the loss with respect to the outputs (softmax probability minus the one-hot label) is passed back layer by layer, multiplied by each layer's weights and by the ReLU derivative, and gives the gradient of the loss with respect to every weight in one backward pass. Adam then uses these gradients to update the weights.

**Split.** Consecutive scans are 0.5 s apart and almost identical, so a random split would put near-copies of the same scan in both training and test sets and overstate accuracy. The data were therefore split in blocks of 40 consecutive scans (20 s), 70/15/15 % for training, validation and test (4080 / 880 / 903 scans).

**Results.** Test accuracy is 90.7 %, against 44.3 % for always guessing the majority class. Clear and wall are both recognised with F1 = 0.92. Box is the weakest class (precision 0.764, recall 0.818): it is the rarest (408 of 5,863 samples), and it is most often confused with wall. The distance table explains why. Accuracy is above 94 % close to an object and 92–100 % far away, but drops to 75 % at 0.8–1.2 m. At 1 m a 0.30 m box covers only about 17 degrees, i.e. about four of the 46 input beams (spaced 4 degrees apart), so a box face looks very much like a short piece of wall.

**Problems found and fixed.** In the first long recording the wander controller sometimes got stuck spinning in place in a corner, which produced hundreds of near-identical scans; runs of scans taken within 3 cm of each other were capped at 8, removing 1,338 scans. A filter based on the difference between consecutive scans did not work, because LiDAR noise makes every pair of scans slightly different, so the pose-based filter was used instead. An earlier 20-minute dataset gave 94.2 % test accuracy, but with only 22 box samples in its test set that figure was too noisy to trust, which is why more data were collected.

| True \ predicted | clear | box | wall | Precision | Recall |
|---|---|---|---|---|---|
| clear | 373 | 14 | 13 | 0.912 | 0.932 |
| box | 8 | 81 | 10 | 0.764 | 0.818 |
| wall | 28 | 11 | 365 | 0.941 | 0.903 |

*Test accuracy 90.7 % (always guessing the majority class: 44.3 %). 903 test scans from held-out blocks.*

| Distance to object ahead | Test accuracy (%) | Scans |
|---|---|---|
| 0.0 to 0.5 m | 99.4 | 162 |
| 0.5 to 0.8 m | 94.6 | 149 |
| 0.8 to 1.2 m | 75.0 | 192 |
| 1.2 to 2.0 m | 92.0 | 338 |
| 2.0 to 3.6 m | 100.0 | 62 |

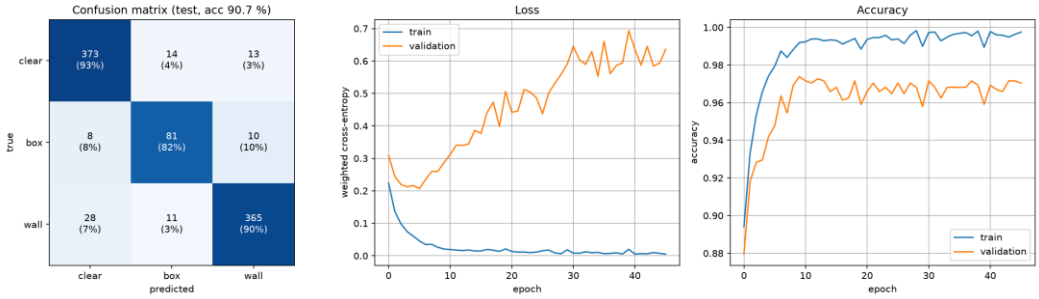



**Figure 8:** *Confusion matrix on the test blocks and training curves (loss and accuracy).*

## 3.4 Centralised versus distributed coordination

Three architectures run the same agreement task: centralised (a star, where followers send their state to a leader that broadcasts the average, so 2 rounds), distributed on a ring, and distributed on a complete graph. As in Webots, one round is 0.2 s, and the team counts as converged when every robot is within 1 cm of the others. This is a round-based simulation of the same update rule used in the Webots consensus run (it needs 5 rounds for N = 4, against 6.1 rounds measured in Webots; the simulation uses ε = 1/3 and Webots ε = 0.3, with contraction factors of 0.33 and 0.40, so the difference is expected), so it extends that result to team sizes that were not run in the simulator.

**Scaling.** For a ring, $\lambda_2 = 2 - 2\cos(2\pi/N) \approx (2\pi/N)^2$, so the number of rounds grows roughly as $N^2$: from 5 rounds (1 s) at N = 4 to 795 rounds (159 s) at N = 64. The star always needs 2 rounds, but the leader must receive N − 1 messages every round (63 at N = 64), while a ring robot receives only 2 whatever the team size. The complete graph agrees in one round, but its message count grows as N(N − 1), 4,032 per round at N = 64, which a shared radio channel would not support.

**Failure.** In an eight-robot test where one robot stops after round 3, the result depends on which robot fails. If the leader fails, the others never agree, because nobody else computes the average. If a follower fails, the rest still agree after the next exchange. If a ring robot fails, the remaining robots still agree, but on a point about 9 cm (0.092 m) away from the true average of the survivors, because the dead robot's state had already been mixed into theirs before it failed. Distributed coordination is therefore slower but has no single point of failure, while centralised coordination is fast and cheap but fails completely if the leader is lost.

| N | Star: rounds / msgs | Ring: rounds / time (s) / msgs | Complete: rounds / msgs |
|---|---|---|---|
| 4 | 2 / 6 | 5 / 1.0 / 40 | 1 / 12 |
| 8 | 2 / 14 | 22 / 4.4 / 352 | 1 / 56 |
| 16 | 2 / 30 | 67 / 13.4 / 2144 | 1 / 240 |
| 32 | 2 / 62 | 341 / 68.2 / 21824 | 1 / 992 |
| 64 | 2 / 126 | 795 / 159.0 / 101760 | 1 / 4032 |

*One round = 0.2 s; converged = every robot within 1 cm of the others. Values from report/central_vs_distributed.txt.*

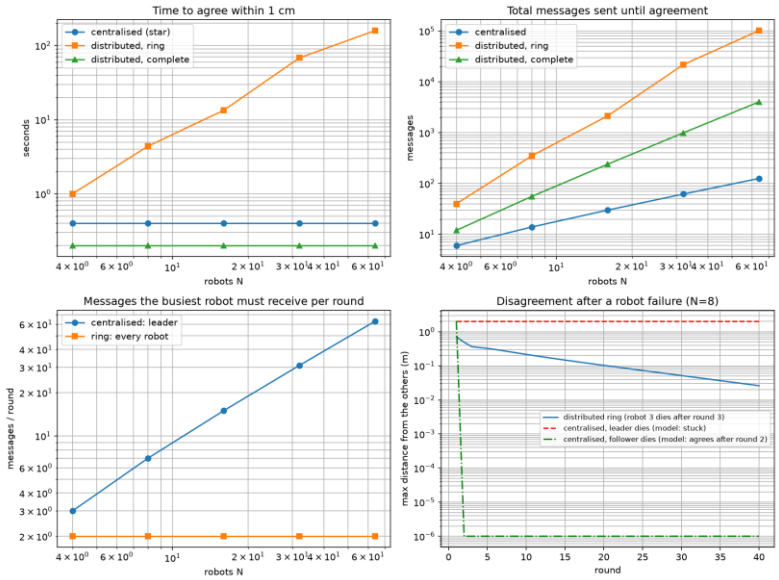



**Figure 9:** *Time to agree, total messages, load on the busiest robot and behaviour after a robot failure.*

## 4. Ethical considerations (South African context)

**Privacy.** The Protection of Personal Information Act 4 of 2013 (POPIA) requires, among other conditions, that personal information is collected for a specific and explicitly defined purpose (purpose specification, section 13), that processing is adequate, relevant and not excessive (minimality, section 10) and that appropriate security safeguards are in place (section 19). A LiDAR scan records only distances and does not identify a person, but a camera image can. A deployed version of this system should therefore process camera images on the robot, keep only the derived labels (box, wall, person present) and the occupancy map, and discard raw frames immediately; any images that must be kept for debugging should be encrypted, access-controlled and deleted on a fixed schedule.

**Employment.** South Africa's official unemployment rate was 33.6 % in the second quarter of 2026, with 8.5 million people unemployed and youth (15–34) unemployment at 47.4 % (SAnews, 2026). Cheap teams of small robots for warehouse logistics, stock-taking or inspection could displace exactly the low-skill jobs that are scarce. The better path is augmentation: robots take the repetitive, dangerous or dirty parts of a task (inspecting confined spaces, carrying loads), people supervise, maintain and handle exceptions, and deployment is tied to reskilling programmes so that the people affected move into those roles.

**Safety and liability.** The Occupational Health and Safety Act 85 of 1993 places a duty on employers to provide a workplace that is safe and without risk to health, as far as reasonably practicable (section 8). The Consumer Protection Act 68 of 2008 makes producers, importers, distributors and retailers liable for harm caused by unsafe or defective goods regardless of negligence (section 61). This project's own numbers show why the

learned components cannot be the only safety layer: the classifier is only 75 % accurate at 0.8–1.2 m, exactly the range at which a robot must decide whether to stop, and the path error reached 7 cm at corners. A real deployment would need an independent, simple LiDAR emergency stop, speed limits near people, and clear records of who is responsible when something goes wrong.

**Reliability and infrastructure.** Load-shedding and uneven connectivity mean that a system depending on a central server or a constant network link may fail without warning. The failure experiment in 3.4 shows the same risk inside the team: losing the leader stops the centralised team from agreeing at all, while the distributed ring keeps working with a small error. This favours on-board processing and distributed coordination, with battery-aware behaviour that brings robots to a safe stop when power or links are lost.

**Bias and representativeness.** The classifier was trained in a 4 x 4 m arena with flat floors, good lighting and six identical boxes. It has never seen people, uneven ground, informal-settlement layouts, farm equipment or mine tunnels, and nothing in the results says it would generalise there. Data must be collected in the places where the system will be used, and accuracy should be reported separately for each setting, so that the system is not quietly worse in poorer or less-studied environments.

**Dual use and accountability.** The same coordination and sensing methods could be repurposed for surveillance, crowd monitoring or policing. Decisions about deployment should be made openly by the organisation that is accountable for the outcome, with the affected workers and communities consulted, a human able to override the robots at all times, and honest reporting of limitations. This report tries to do the last of these by publishing the full confusion matrix, the distance range where the classifier is weak and the conditions under which the team fails to agree.

## 5. Limitations and conclusion

**What worked.** Fusing the compass heading with wheel distance removed a 38-degree heading drift and gave a clean occupancy map. A* matched Dijkstra's optimal path lengths while expanding 7–32 times fewer cells. The tuned PD heading controller settled a 90-degree turn in 0.90 s with no overshoot, and the full closed loop in Webots followed the A* path with a mean cross-track error of 1.4 cm, agreeing with the offline simulation to within 0.4 s and 0.1 cm. Four robots reached consensus within 1 cm in 1.2 s (6 rounds); improved PSO found the optimal allocation in half of the runs with a mean cost 4 % above the optimum; and the LiDAR classifier reached 90.7 % test accuracy against a 44.3 % baseline.

**Limitations.** All results come from simulation, where compass and wheel noise are small and there is little wheel slip, so the real Burger would perform worse. The wander controller sometimes spun in place, which forced data cleaning. The box class is small (408 samples), the arena is small and uniform, and the allocation problem is small enough to solve by brute force.

**Next steps.** Add an independent LiDAR emergency stop below the learned classifier; smooth the planned path to remove corner errors; test the pipeline on a real TurtleBot3; and combine the PSO allocation with an obstacle-aware multi-robot planner so that the routes the team agrees on are also collision-free.

## References


Consumer Protection Act 68 of 2008. Republic of South Africa, Government Gazette 32186.

Cyberbotics (2025) *Webots Reference Manual, R2025a*. Available at: https://cyberbotics.com/doc/reference/

Dijkstra, E.W. (1959) 'A note on two problems in connexion with graphs', *Numerische Mathematik*, 1, pp. 269–271.

Hart, P.E., Nilsson, N.J. and Raphael, B. (1968) 'A formal basis for the heuristic determination of minimum cost paths', *IEEE Transactions on Systems Science and Cybernetics*, 4(2), pp. 100–107.

Kennedy, J. and Eberhart, R. (1995) 'Particle swarm optimization', *Proceedings of ICNN'95, International Conference on Neural Networks*, Perth, vol. 4, pp. 1942–1948.

Occupational Health and Safety Act 85 of 1993. Republic of South Africa, Government Gazette 14918.

Olfati-Saber, R., Fax, J.A. and Murray, R.M. (2007) 'Consensus and cooperation in networked multi-agent systems', *Proceedings of the IEEE*, 95(1), pp. 215–233.

Protection of Personal Information Act 4 of 2013. Republic of South Africa, Government Gazette 37067.

Rumelhart, D.E., Hinton, G.E. and Williams, R.J. (1986) 'Learning representations by back-propagating errors', *Nature*, 323, pp. 533–536.

SAnews (2026) 'SA official unemployment rate rises to 33.6%', 11 August. Reporting Statistics South Africa, *Quarterly Labour Force Survey, Quarter 2: 2026*. Available at: https://www.sanews.gov.za/south-africa/sa-official-unemployment-rate-rises-336

Shi, Y. and Eberhart, R. (1998) 'A modified particle swarm optimizer', *Proceedings of the IEEE International Conference on Evolutionary Computation*, Anchorage, pp. 69–73.

Thrun, S., Burgard, W. and Fox, D. (2005) *Probabilistic Robotics*. Cambridge, MA: MIT Press.